

OSNOVE UMETNE INTELIGENCE

2025/26

linearna regresija
nevronske mreže
nenadzorovano učenje

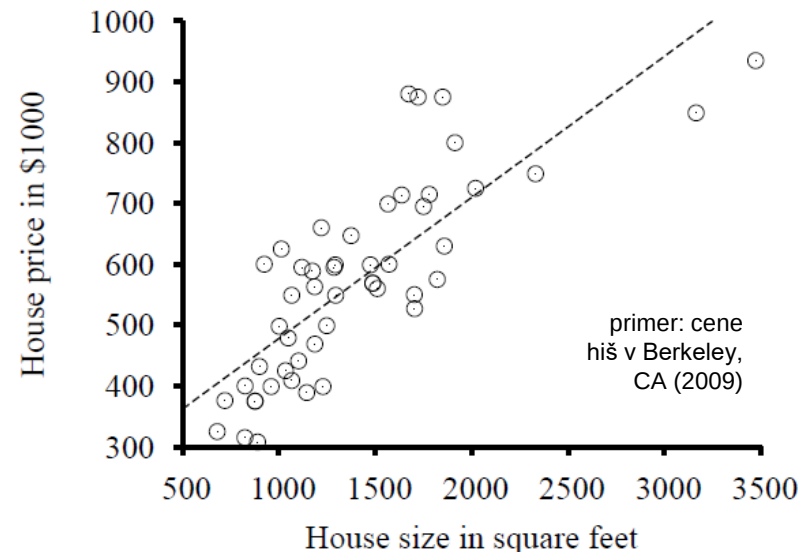
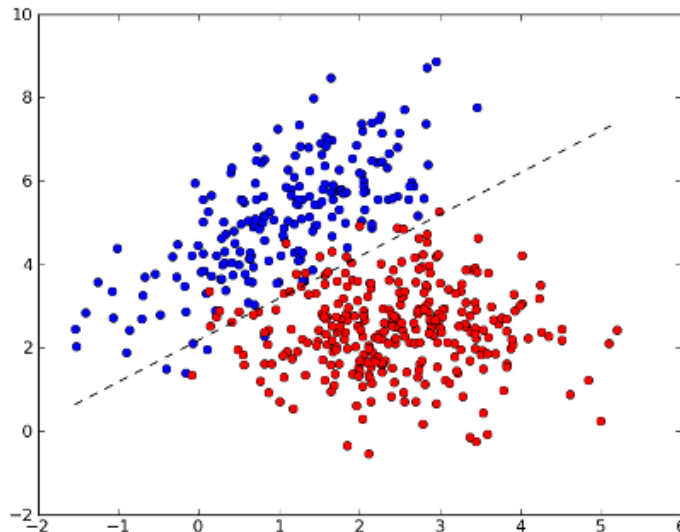
Linearni modeli

- uporaba pri **klasifikaciji** (kot separator razredov) in **regresiji** (kot prileganje skozi podane točke)
- linearni model z **eno odvisno** spremenljivko (angl. *univariate linear model*):

$$h(x) = w_1x + w_0$$

w_0 in w_1 sta **uteži** (angl. *weights*) spremenljivk (koeficienta)

- **linearna regresija**: postopek iskanja funkcije $h(x)$ (oziroma uteži w_0 in w_1), ki se najboljše prilega učnim podatkom



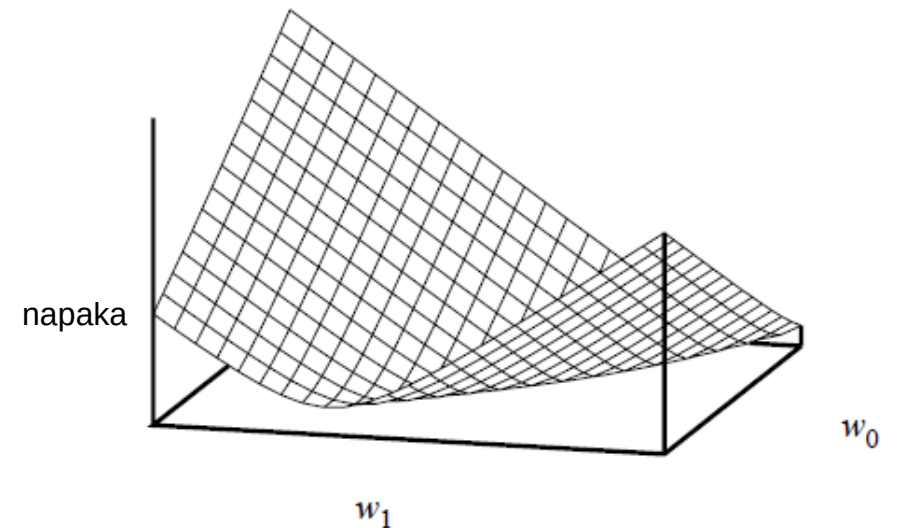
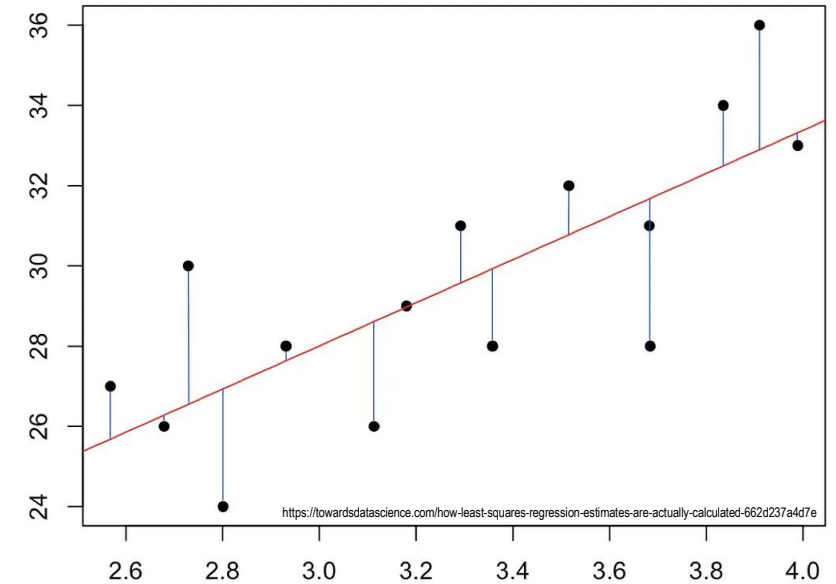
Linearna regresija

- optimizacijo izvedemo z minimizacijo srednje kvadratne napake:

$$\text{napaka}(h) = \sum_{j=1}^N \left(y_j - (w_1 x_j + w_0) \right)^2$$

- prostor koeficientov je konveksen, lokalni minimumi ne obstajajo (samo globalni)
- obstaja analitična rešitev:

$$w_1 = \frac{N(\sum x_j y_j) - (\sum x_j)(\sum y_j)}{N(\sum x_j^2) - (\sum x_j)^2}$$
$$w_0 = \frac{\sum y_j - w_1(\sum x_j)}{N}$$



Posplošitev v več dimenzij

- možna je posplošitev v **višje število dimenzij** – več neodvisnih spremenljivk (atributov) (angl. *multivariate linear regression*)

$$h(x) = w_0 + \sum_i w_i x_{j,i}$$

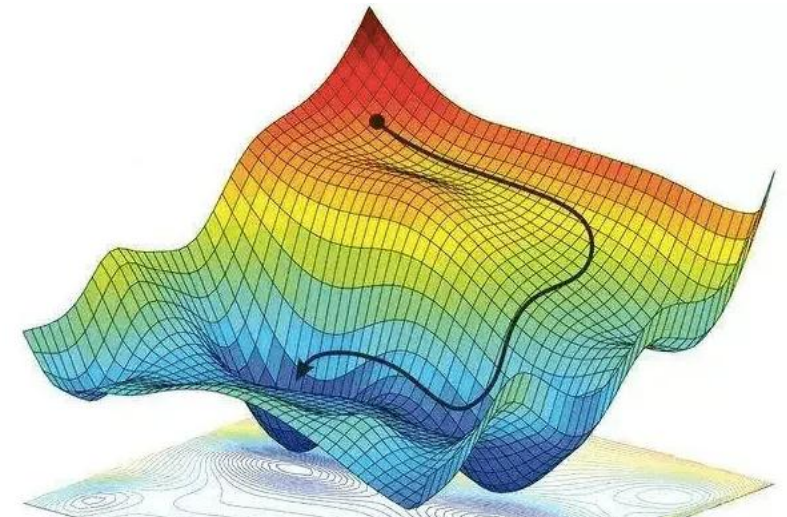
kjer so w_i uteži (koeficienti), $x_{j,i}$ pa i -ta spremenljivka (atribut) primera x_j

- uteži lahko določimo **analitično**: $\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$
kjer je $\mathbf{X}$ matrika s podatki (vrstice – učni primeri, stolpci – atributi), $\mathbf{y}$ pa vektor z vrednostmi odvisnih spremenljivk primerov
- v praksi se odločamo za iskanje koeficientov z **gradientnim spustom** ($\mathbf{w}$ – vektor uteži, η – hitrost učenja)

$\mathbf{w} \leftarrow$ naključna začetna rešitev
ponavljaj do konvergence

za vsak w_i v $\mathbf{w}$:

$$w_i \leftarrow w_i - \eta \frac{\partial \text{napaka}(\mathbf{w})}{\partial w_i}$$



Linearni modeli pri klasifikaciji

- možnih ustreznih linearnih modelov je več, odvisni so od vrednosti uteži:

$$h_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

- preprosto iskanje rešitve – **stohastični gradientni spust** s posodabljanjem uteži za vsak učni primer $(\mathbf{x}, y) = (x_1, x_2, \dots, x_n, y)$:

$$w_i \leftarrow w_i + \eta \cdot (y - h(\mathbf{x})) \cdot x_i$$

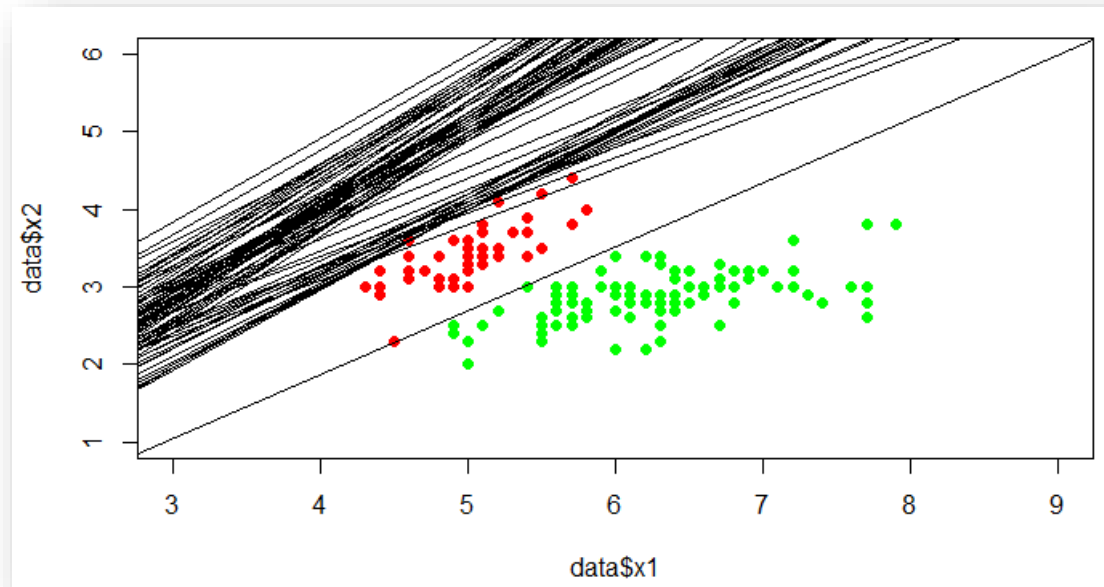
- w_i - utež, η – hitrost učenja (> 0), y – ciljna vrednost, $h(\mathbf{x})$ - izhodna vrednost za učni primer $(\mathbf{x}, y)$ pri trenutni uteži, x_i - vrednost i-tega atributa učnega primera

		vrednost člena ($y - h(\mathbf{x})$)	vrednost produkta ($y - h(\mathbf{x})$) $\cdot x_i$	rezultat
$y = h(\mathbf{x})$	izhod je enak ciljni vrednosti	0	ni popravka	utež ostane enaka
$y > h(\mathbf{x})$	ciljna vrednost je višja od trenutne izhodne, potreben je popravek $h(\mathbf{x})$ navzgor	> 0	$\begin{cases} > 0 & \text{če } x_i > 0 \\ < 0 & \text{če } x_i < 0 \end{cases}$	utež povečamo za pozitivne in zmanjšamo za negativne x_i
$y < h(\mathbf{x})$	ciljna vrednost je nižja od trenutne izhodne, potreben je popravek $h(\mathbf{x})$ navzdol	< 0	$\begin{cases} < 0 & \text{če } x_i > 0 \\ > 0 & \text{če } x_i < 0 \end{cases}$	utež zmanjšamo za pozitivne in povečamo za negativne x_i

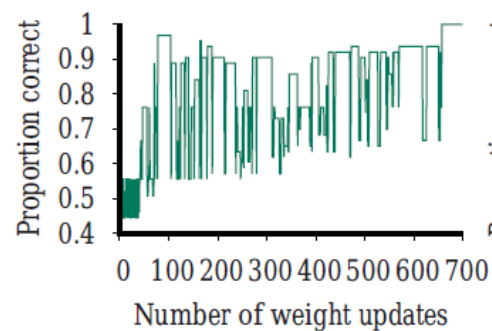
- algoritem lahko pri ustreznem η najde optimalno rešitev tudi za linearno neločljive podatke

Linearni modeli pri klasifikaciji

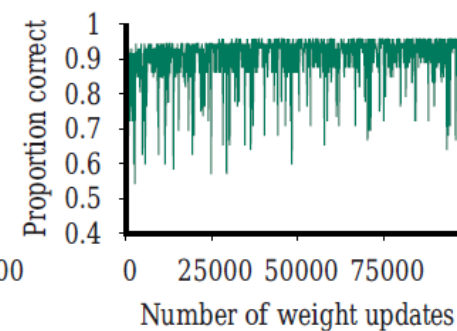
- demo



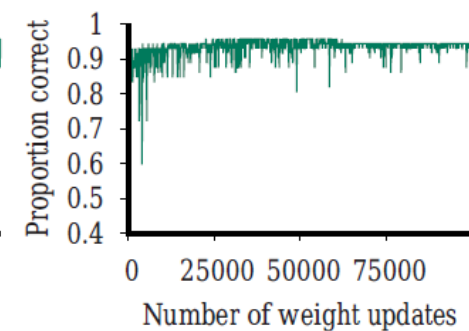
konvergenca algoritma pri linearno ločljivih podatkih (levo) in linearno neločljivih podatkih (sredina); uporaba padajoče hitrosti učenja η (desno)



(a)



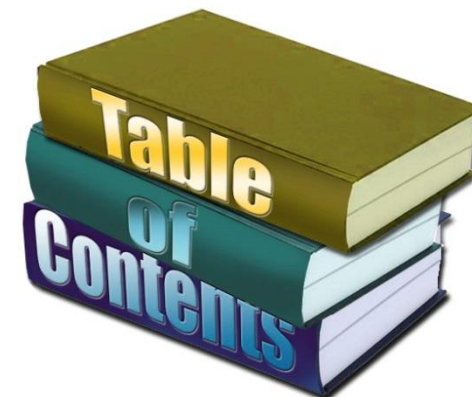
(b)



(c)

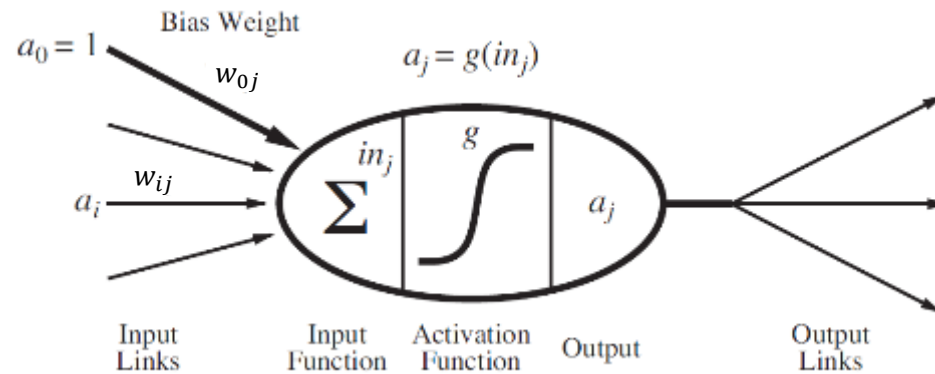
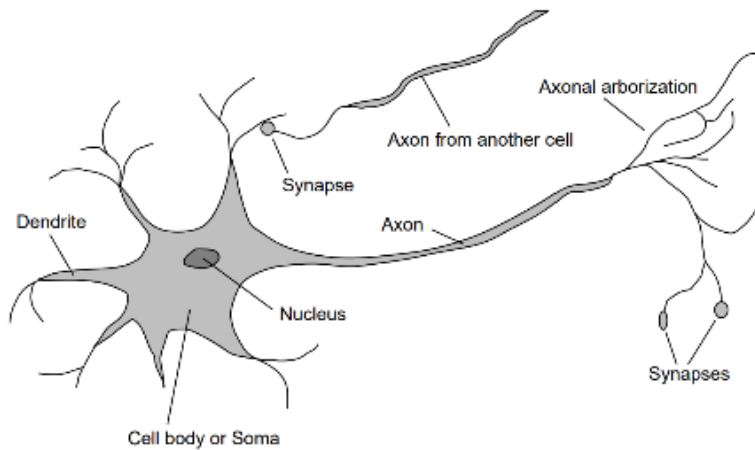
Pregled

- strojno učenje
 - uvod v strojno učenje
 - učenje odločitvenih dreves
 - učenje dreves iz šumnih podatkov (rezanje dreves)
 - ocenjevanje učenja
 - diskretizacija atributov, obravnava manjkajočih vrednosti
 - naivni Bayesov klasifikator
 - nomogrami
 - k najbližjih sosedov
 - lokalno utežena regresija
 - regresijska drevesa
 - linearna regresija
 - nevronske mreže
 - nenadzorovano učenje

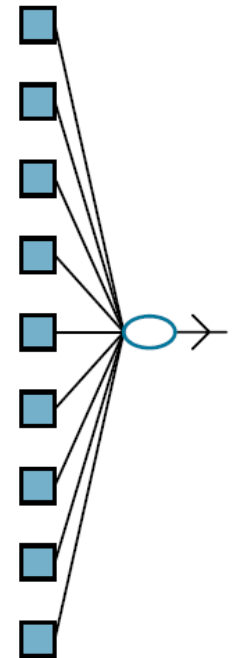


Nevronske mreže

- motivacija: aktivnosti v človeških možganih se pretežno izvajajo z elektro-kemičnimi procesi
- izdelava umetnih nevronske mreže z nevroni, ki oponašajo biološke nevrone (možganske celice)
- McCulloch in Pitts (1943) predlagata preprost matematičen model nevrona
- umetni nevron izračuna **uteženo linearno kombinacijo** vhodov in jo z aktivacijsko funkcijo preslika v izhodno vrednost

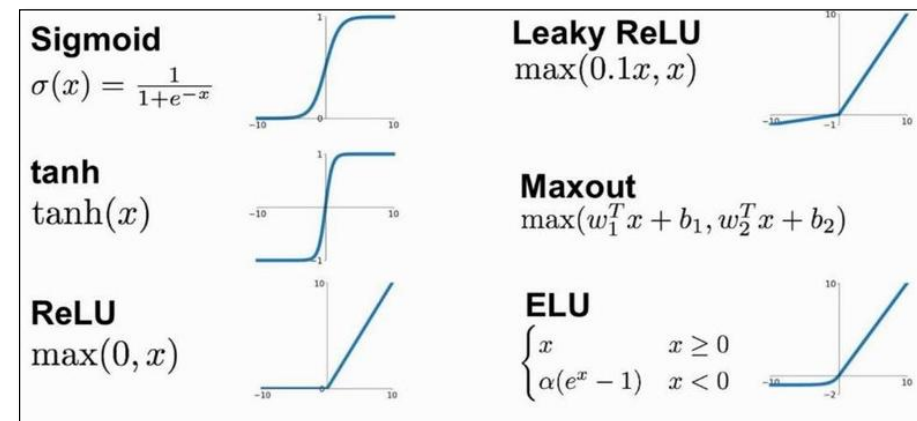


$$a_j = g\left(\sum_i w_{ij}a_i\right) = g(z_i)$$

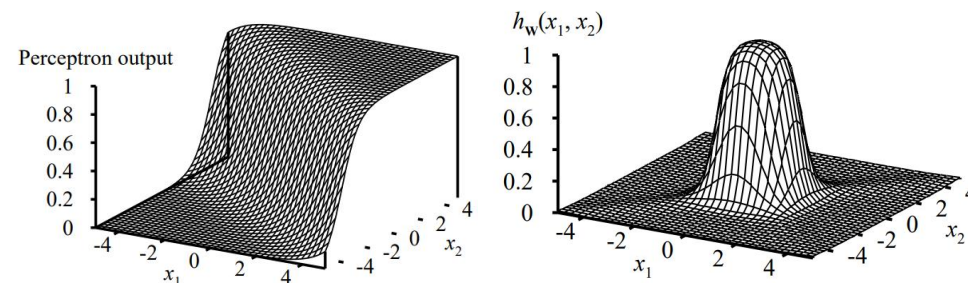
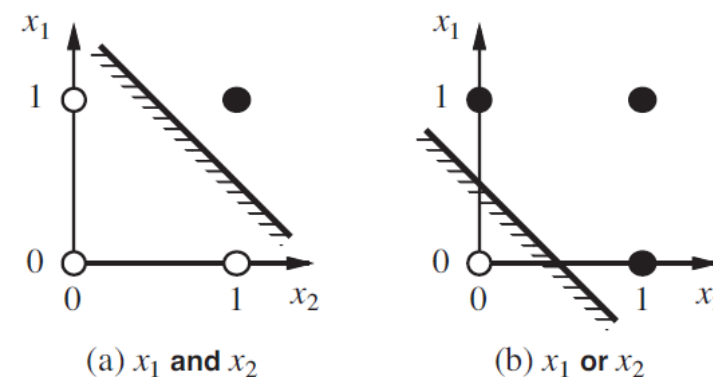


Aktivacijska funkcija, mreža

- cilj aktivacijske funkcije: zagotoviti, da lahko cela nevronska mreža zagotavlja nelinearno funkcijo
- **en nevron** implementira linearno funkcijo v problemskem prostoru
 - lahko se uporabi za **regresijo** ali za **klasifikacijo** (ločevanje dveh razredov)
 - možna implementacija preprostih logičnih in drugih funkcij
 - spreminjanje uteži spreminja lokacijo/orientacijo funkcije
- **nevronska mreža** je množica nevronov, ki so medsebojno povezani v izbrano topologijo
 - izračunavanje bolj kompleksne funkcije – kombinacij funkcij različnih nevronov
 - en velik skriti nivo omogoča implementacijo poljubne zvezne funkcije
 - z dvema nivojema je možna tudi implementacija nezveznih funkcij
- **demo:** [Tensorflow - A Neural Network Playground](#)

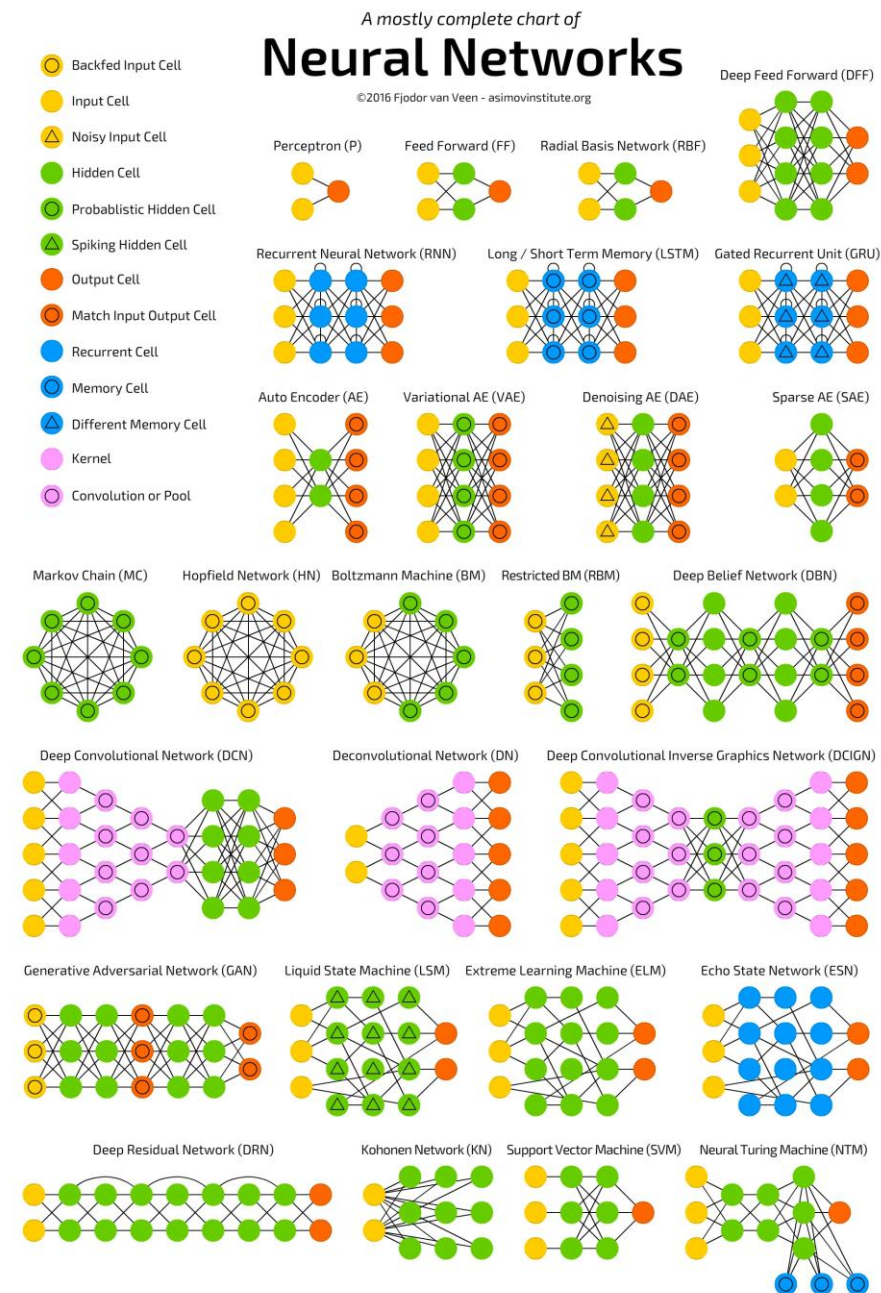


https://www.researchgate.net/figure/fig3-The-basic-activation-functions-of-the-neural-networksNeural-Networks_fig3_350567223



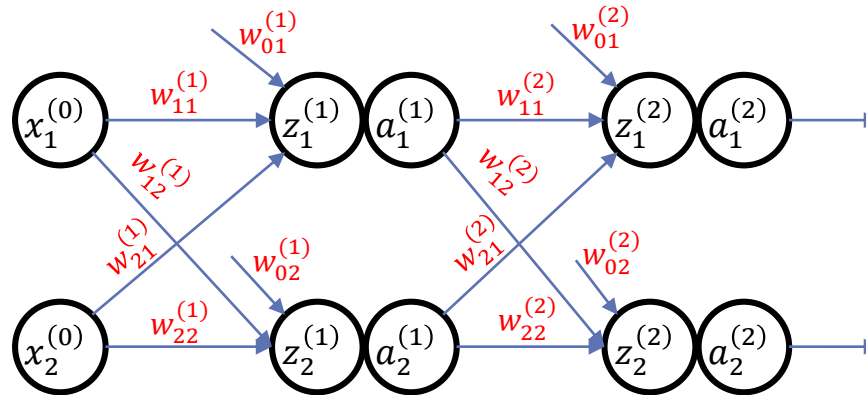
Različne implementacije mrež

- **feed-forward network**: aciklične povezave samo v smeri od vhoda proti izhodu (samo implementirajo izbrano funkcijo, ne hranijo notranjega stanja)
 - nevroni **organizirani v plasti** (ena plast, več plasti, različna števila skritih plasti in nevronov, globoko učenje)
 - možen **en izhodni nevron** (za napoved ene zvezne vrednosti) ali **več izhodov** (za več hkratnih bitov – npr. pri seštevanju bitov: vsota in prenos, klasifikacija števk 0-9 itd.)
- **rekurenčna mreža**: uporablja svoje izhode kot ponovne vhode v mrežo (implementacija dinamičnega sistema, ki hrani notranje stanje – to je odvisno od preteklih izhodov)
 - implementacija kratkoročnega spomina podobno kot v možganih, vendar težje razumevanje/analiza delovanja
- **konvolucijske mreže** (uporabne v računalniškem vidu)



Primer napovedi

- primer nevronske mreže z dvema vhodoma, enim skritim nivojem z dvema nevronoma in dvema izhodoma



- denimo, da velja: $g(z) = \text{ReLU}(z) = \max(0, z)$, $x = (1, 2)$, $w_{ij}^{(1)} = \begin{bmatrix} 0.1 & 0.2 \\ 0.4 & 0.1 \\ 0.5 & 0.5 \end{bmatrix}$, $w_{ij}^{(2)} = \begin{bmatrix} 0.1 & 0.1 \\ 0.2 & 0.5 \\ 0.8 & 0.1 \end{bmatrix}$
- zgornja mreža izračunava funkcijo:

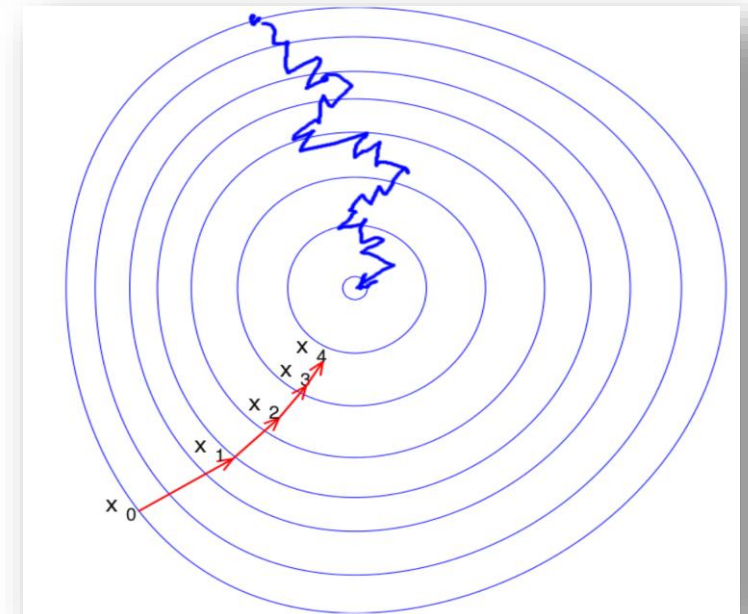
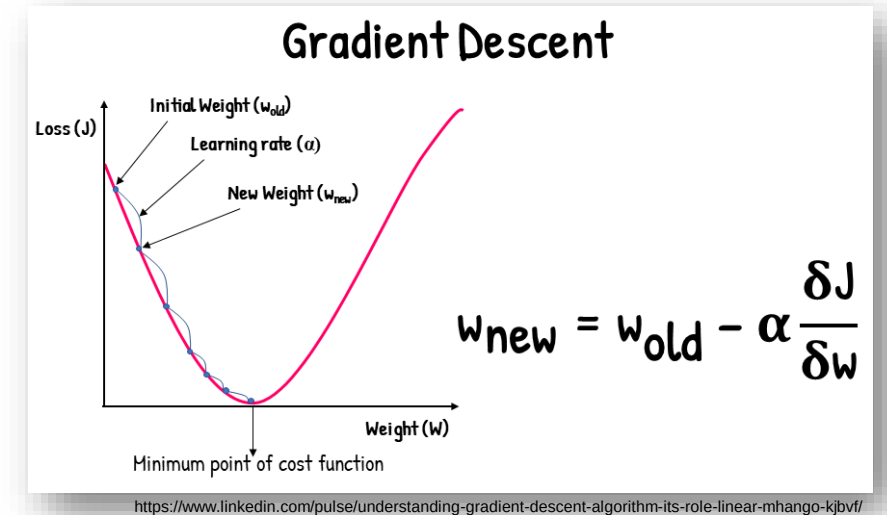
$$\begin{aligned}
 a_1^{(2)} &= g\left(w_{01}^{(2)} + w_{11}^{(2)} a_1^{(1)} + w_{21}^{(2)} a_2^{(1)}\right) = \\
 &= g\left(w_{01}^{(2)} + w_{11}^{(2)} g\left(w_{01}^{(1)} + w_{11}^{(1)} x_1^{(0)} + w_{21}^{(1)} x_2^{(0)}\right) + w_{21}^{(2)} g\left(w_{02}^{(1)} + w_{12}^{(1)} x_1^{(0)} + w_{22}^{(1)} x_2^{(0)}\right)\right) \\
 &= g\left(0.1 + 0.2 \cdot g(0.1 + 0.4 \cdot 1 + 0.5 \cdot 2) + 0.8 \cdot g(0.2 + 0.1 \cdot 1 + 0.5 \cdot 2)\right) = 1.44 \\
 a_2^{(2)} &= ?
 \end{aligned}$$

Vzratno razširjanje napake

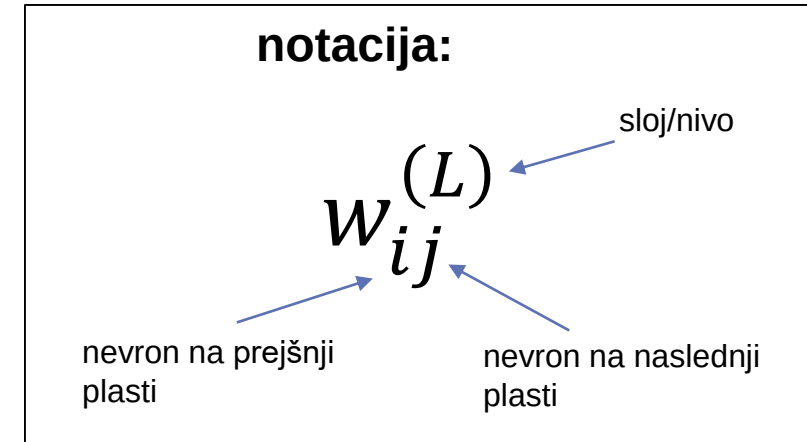
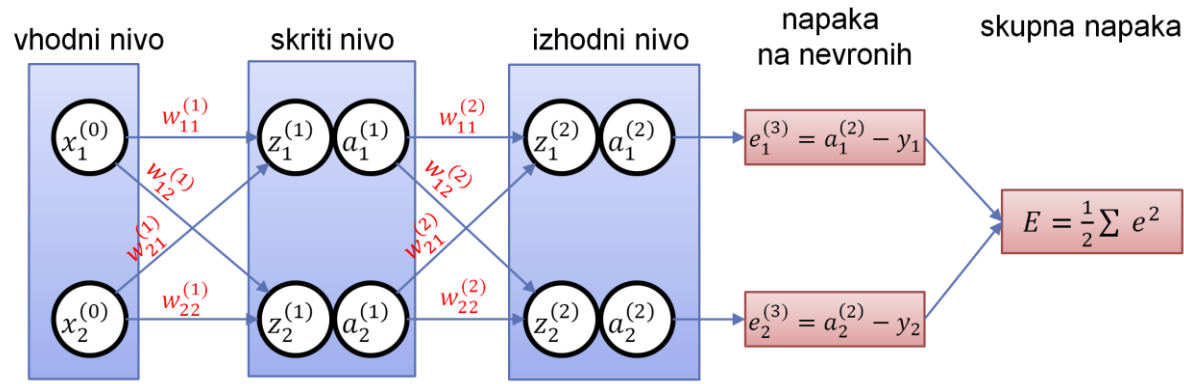
1. **inicializiraj** uteži (ponavadi majhne neničelne naključne vrednosti)
2. **izračunaj napovedi** za učni/e primer/e
3. **izračunaj izgubo** (funkcijo napake na izhodnih nevronih; npr. srednja kvadratna napaka, križna entropija)
4. **vzratno razširi napako** od izhoda proti vhodu:
 - izračunaj gradient napake za izhodni nivo
 - izračunaj gradient napake za enega ali več skritih nivojevzgornji postopek uporabi bodisi
 - na majhni podmnožici učnih primerov (mini-batch gradientni spust)
 - na celotni učni množici (gradientni spust)
 - na vsakem posameznem učnem primeru (stohastični gradientni spust)
5. za vsako posamezno utež w **uporabi obliko gradientnega spusta**, da popraviš vrednost uteži s **hitrostjo učenja** η :

$$W \leftarrow W - \eta \cdot \frac{\partial E}{\partial w}$$

6. **ponovi korake 2-5** do ustavitvenega kriterija, npr.:
 - izbrano število epoh (celotnih ponovitev skozi vse učne primere),
 - do znižanja napake do želene meje.

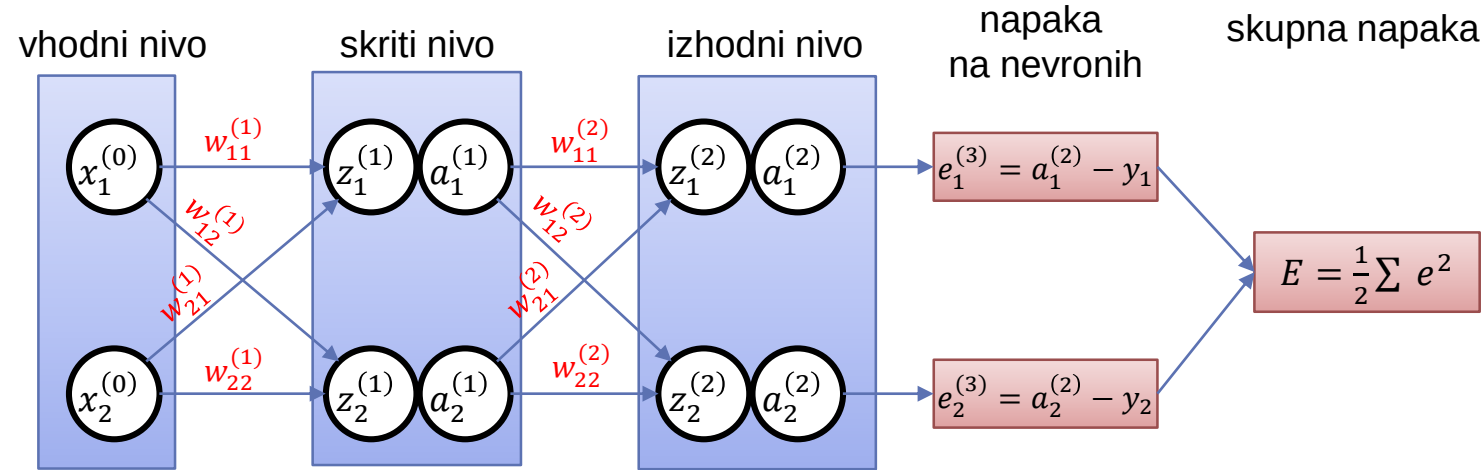


Učenje nevronske mreže



- za dani učni primer izračunamo aktivacije nevronov na izhodni plasti in jih primerjamo s ciljnim vrednostmi, da ovrednotimo napako (funkcijo izgube) na nevronu: $E = \frac{1}{2} \sum_k (a_i^{(L)} - y_i)^2$
- cilj: kako nastaviti **vse uteži** v nevronske mreži, da bo ta napaka minimalna: zanima nas torej vpliv spremembe uteži na spremembo napake, kar je ravno: $\frac{\delta E}{\delta w}$
- ugotovimo: napaka i -tega izhodnega nevrone $e_i^{(L)}$ je odvisna od izhoda (aktivacije) nevrone $a_i^{(L-1)}$, ta je odvisna od vhoda v aktivacijsko funkcijo $z_i^{(L-1)}$, ta pa je odvisna od aktivacij nevronov na prejšnji plasti in uteži $w_{jk}^{(L-1)}$

Učenje nevronske mreže



- začnimo z nastavljanjem uteži na zadnjem nivoju, tik pred izhodnim ($w_{11}^{(2)}, w_{12}^{(2)}, w_{21}^{(2)}, w_{22}^{(2)}$), in uporabimo **verižno pravilo za odvajanje**:

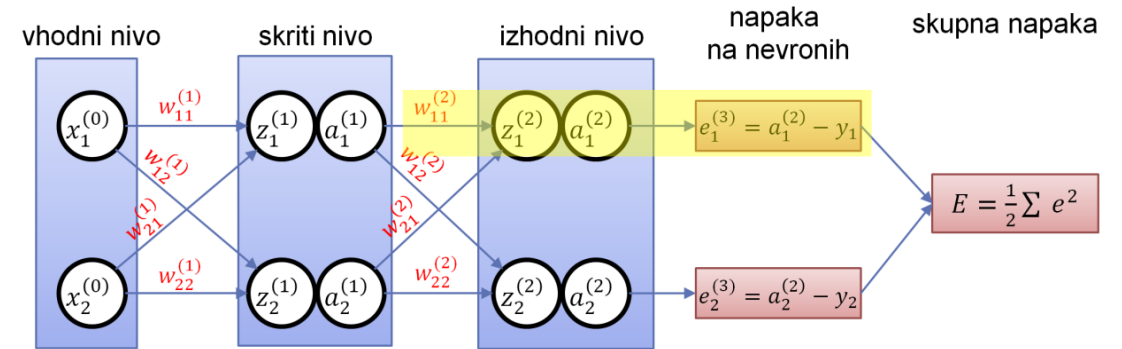
$$\frac{\partial E}{\partial w_{11}^{(2)}} = \frac{\partial E}{\partial a_1^{(2)}} \cdot \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \cdot \frac{\partial z_1^{(2)}}{\partial w_{11}^{(2)}}$$

$$\frac{\partial E}{\partial w_{12}^{(2)}} = \frac{\partial E}{\partial a_2^{(2)}} \cdot \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \cdot \frac{\partial z_2^{(2)}}{\partial w_{12}^{(2)}}$$

$$\frac{\partial E}{\partial w_{21}^{(2)}} = \frac{\partial E}{\partial a_1^{(2)}} \cdot \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \cdot \frac{\partial z_1^{(2)}}{\partial w_{21}^{(2)}}$$

$$\frac{\partial E}{\partial w_{22}^{(2)}} = \frac{\partial E}{\partial a_2^{(2)}} \cdot \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \cdot \frac{\partial z_2^{(2)}}{\partial w_{22}^{(2)}}$$

Učenje nevronske mreže



- kako izračunati $\frac{\partial E}{\partial w_{11}^{(2)}} = \frac{\partial E}{\partial a_1^{(2)}} \cdot \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \cdot \frac{\partial z_1^{(2)}}{\partial w_{11}^{(2)}}$?

- izračunamo lahko:

$$\frac{\partial E}{\partial a_1^{(2)}} = \frac{\partial \left(\frac{1}{2} \sum (a_1^{(2)} - y_1)^2 \right)}{\partial a_1^{(2)}} = a_1^{(2)} - y_1$$

$$\frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} = \sigma' (z_1^{(2)}) = a_1^{(2)} (1 - a_1^{(2)}) \quad (\text{odvod sigmoidne funkcije})$$

$$\frac{\partial z_1^{(2)}}{\partial w_{11}^{(2)}} = \frac{\partial (w_{01}^{(2)} + \sum_i a_i^{(1)} w_{i1}^{(2)})}{\partial w_{11}^{(2)}} = a_1^{(1)} \quad (a_i \text{ so aktivacije nevronov na prejšnji plasti})$$

- torej velja:

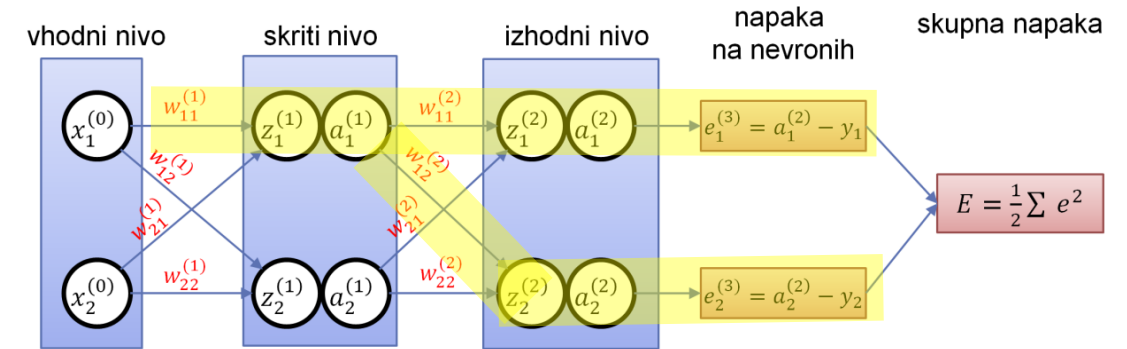
$$\frac{\partial E}{\partial w_{11}^{(2)}} = (a_1^{(2)} - y_1) a_1^{(2)} (1 - a_1^{(2)}) a_1^{(1)}$$

$$\frac{\partial E}{\partial w_{12}^{(2)}} = (a_2^{(2)} - y_2) a_2^{(2)} (1 - a_2^{(2)}) a_1^{(1)}$$

$$\frac{\partial E}{\partial w_{21}^{(2)}} = (a_1^{(2)} - y_1) a_1^{(2)} (1 - a_1^{(2)}) a_2^{(1)}$$

$$\frac{\partial E}{\partial w_{22}^{(2)}} = (a_2^{(2)} - y_2) a_2^{(2)} (1 - a_2^{(2)}) a_2^{(1)}$$

Učenje nevronske mreže



- kako izračunati gradient napake za nevrone v skitem nivoju?

- izhodni nivo:**

$$\frac{\partial E}{\partial w_{11}^{(2)}} = \frac{\partial E}{\partial a_1^{(2)}} \cdot \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \cdot \frac{\partial z_1^{(2)}}{\partial w_{11}^{(2)}} = (a_1^{(2)} - y_1) a_1^{(2)} (1 - a_1^{(2)}) a_1^{(1)}$$

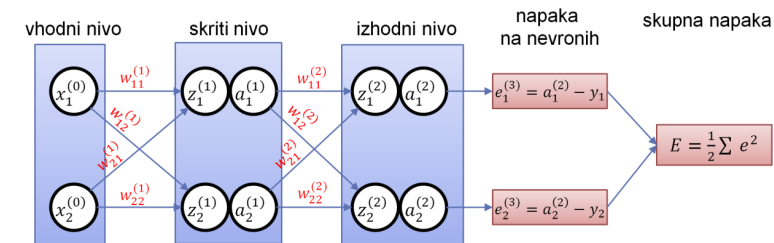
- skriti nivo:**

$$\begin{aligned} \frac{\partial E}{\partial w_{11}^{(1)}} &= \left(\frac{\partial E}{\partial a_1^{(2)}} \cdot \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \cdot \frac{\partial z_1^{(2)}}{\partial a_1^{(1)}} + \frac{\partial E}{\partial a_2^{(2)}} \cdot \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \cdot \frac{\partial z_2^{(2)}}{\partial a_1^{(1)}} \right) \cdot \frac{\partial a_1^{(1)}}{\partial z_1^{(1)}} \cdot \frac{\partial z_1^{(1)}}{\partial w_{11}^{(1)}} \\ &= \left((a_1^{(2)} - y_1) a_1^{(2)} (1 - a_1^{(2)}) w_{11}^{(2)} + (a_2^{(2)} - y_2) a_2^{(2)} (1 - a_2^{(2)}) w_{12}^{(2)} \right) a_1^{(1)} (1 - a_1^{(1)}) x_1^{(0)} \end{aligned}$$

oziroma na splošno za poljubno utež $w_{ij}^{(1)}$ s prve plasti nevronov, k je števec po izhodnih nevronih:

$$\frac{\partial E}{\partial w_{ij}^{(1)}} = \sum_k \left((a_k^{(2)} - y_k) a_k^{(2)} (1 - a_k^{(2)}) w_{jk}^{(2)} \right) a_j^{(1)} (1 - a_j^{(1)}) x_i^{(0)}$$

Vaja



- velja: $g(z) = \sigma(z)$, $x = (1,2)$, $w_{ij}^{(1)} = \begin{bmatrix} 0.1 & 0.2 \\ 0.4 & 0.1 \\ 0.5 & 0.5 \end{bmatrix}$, $w_{ij}^{(2)} = \begin{bmatrix} 0.1 & 0.1 \\ 0.2 & 0.5 \\ 0.8 & 0.1 \end{bmatrix}$

- formule:

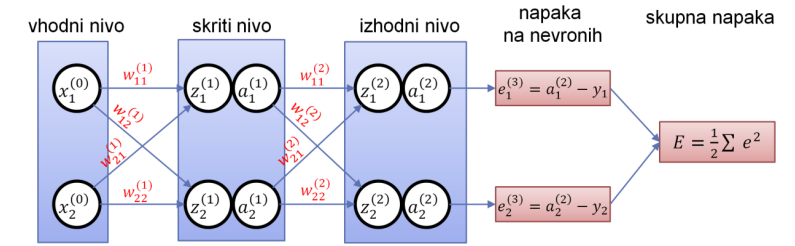
- **izhodni nivo:** $\frac{\partial E}{\partial w_{ij}^{(2)}} = (a_j^{(2)} - y_j) a_j^{(2)} (1 - a_j^{(2)}) a_i^{(1)}$

- **skriti nivo:** $\frac{\partial E}{\partial w_{ij}^{(1)}} = \sum_k \left((a_k^{(2)} - y_k) a_k^{(2)} (1 - a_k^{(2)}) w_{jk}^{(2)} \right) a_j^{(1)} (1 - a_j^{(1)}) x_i^{(0)}$

- Naloga: Za podani vhodni primer $x = (1,2)$ sta izhoda mreže enaka $(a_1^{(2)}, a_2^{(2)}) = (0.713, 0.654)$. Izračunaj gradiente napake za utež $w_{21}^{(2)}$, če je ciljna vrednost izhodnih nevronov $(1.0, 0.5)$.

$$\begin{aligned} \frac{\partial E}{\partial w_{21}^{(2)}} &= (a_1^{(2)} - y_1) a_1^{(2)} (1 - a_1^{(2)}) a_2^{(1)} \\ &= (0.713 - 1.0) \cdot 0.713 \cdot (1 - 0.713) \cdot \sigma(0.2 + 0.1 \cdot 1 + 0.5 \cdot 2) \\ &= (0.713 - 1.0) \cdot 0.713 \cdot (1 - 0.713) \cdot 0.7858 \\ &= -0,046 \end{aligned}$$

Vaja



- Za izračunani gradient iz prejšnje naloge $\frac{\partial E}{\partial w_{21}^{(2)}} = -0,046$ in hitrost učenja $\eta = 0.1$ popravi utež $w_{21}^{(2)}$. Izračunaj novo napoved izhodnega nevrona (za enake podatke kot pri prejšnji nalogi).

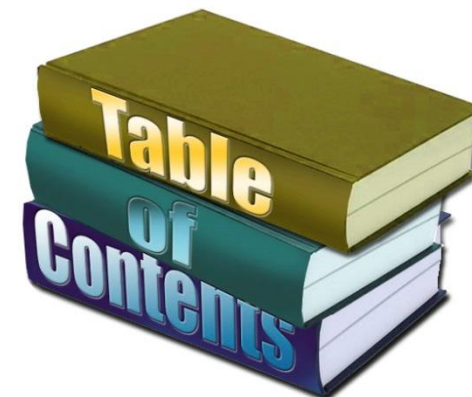
$$w_{21}^{(2)} \leftarrow w_{21}^{(2)} - \eta \cdot \frac{\partial E}{\partial w_{21}^{(2)}}$$

$$w_{21}^{(2)} \leftarrow 0.8 - 0.1 \cdot (-0.046) = 0.8046$$

- Kakšna je nova vrednost prvega izhodnega nevrona? Je bolj ali manj točna v primerjavi s prejšnjo?

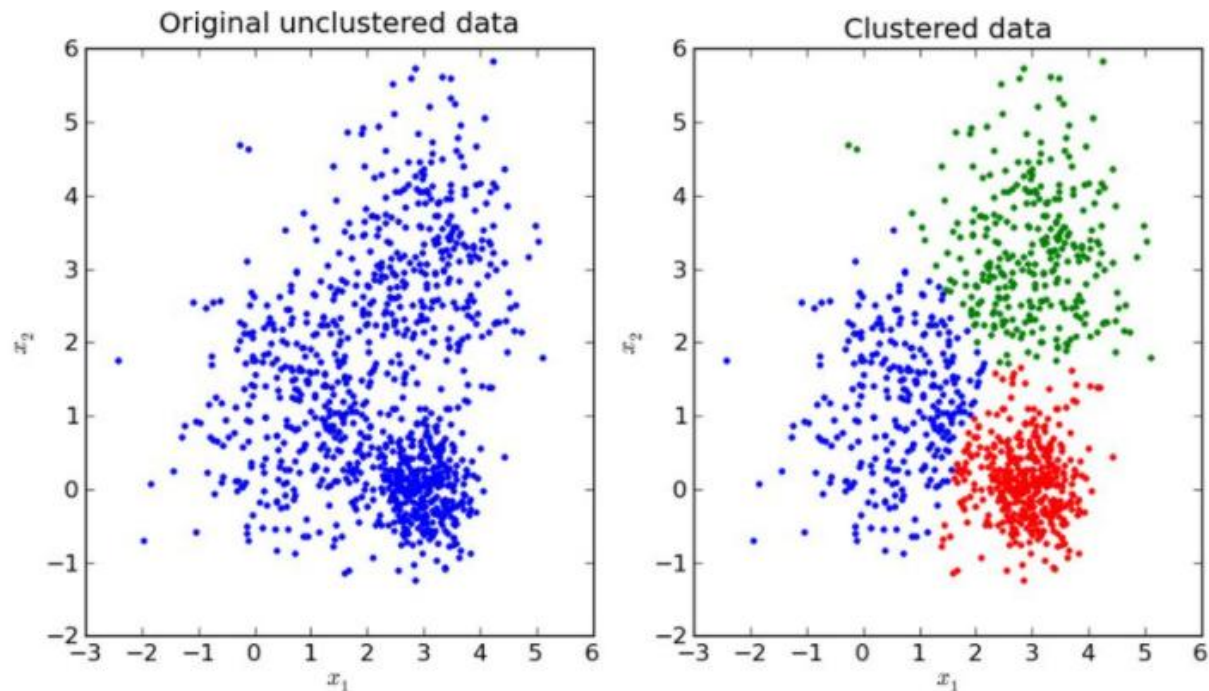
Pregled

- strojno učenje
 - uvod v strojno učenje
 - učenje odločitvenih dreves
 - učenje dreves iz šumnih podatkov (rezanje dreves)
 - ocenjevanje učenja
 - diskretizacija atributov, obravnava manjkajočih vrednosti
 - naivni Bayesov klasifikator
 - nomogrami
 - k najbližjih sosedov
 - lokalno utežena regresija
 - regresijska drevesa
 - linearna regresija
 - nevronske mreže
 - nenadzorovano učenje

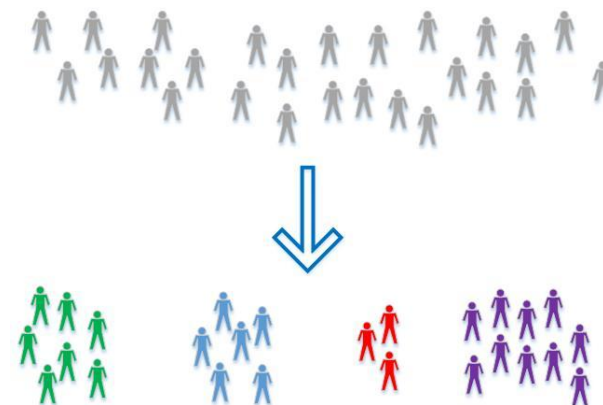


Nenadzorovano učenje

- drugačni scenarij in cilji učenja kot pri nadzorovanem učenju:
 - **nimamo ciljne (odvisne) spremenljivke**, zato nas ne zanima napoved primera
 - podani so samo atributi primerov
- cilj: odkrivanje zakonitosti glede porazdelitve učnih primerov. Vprašanja:
 - ali lahko primere razdelimo v **smiselne skupine**?
 - ali obstaja priročen način za **vizualizacijo** podatkov?



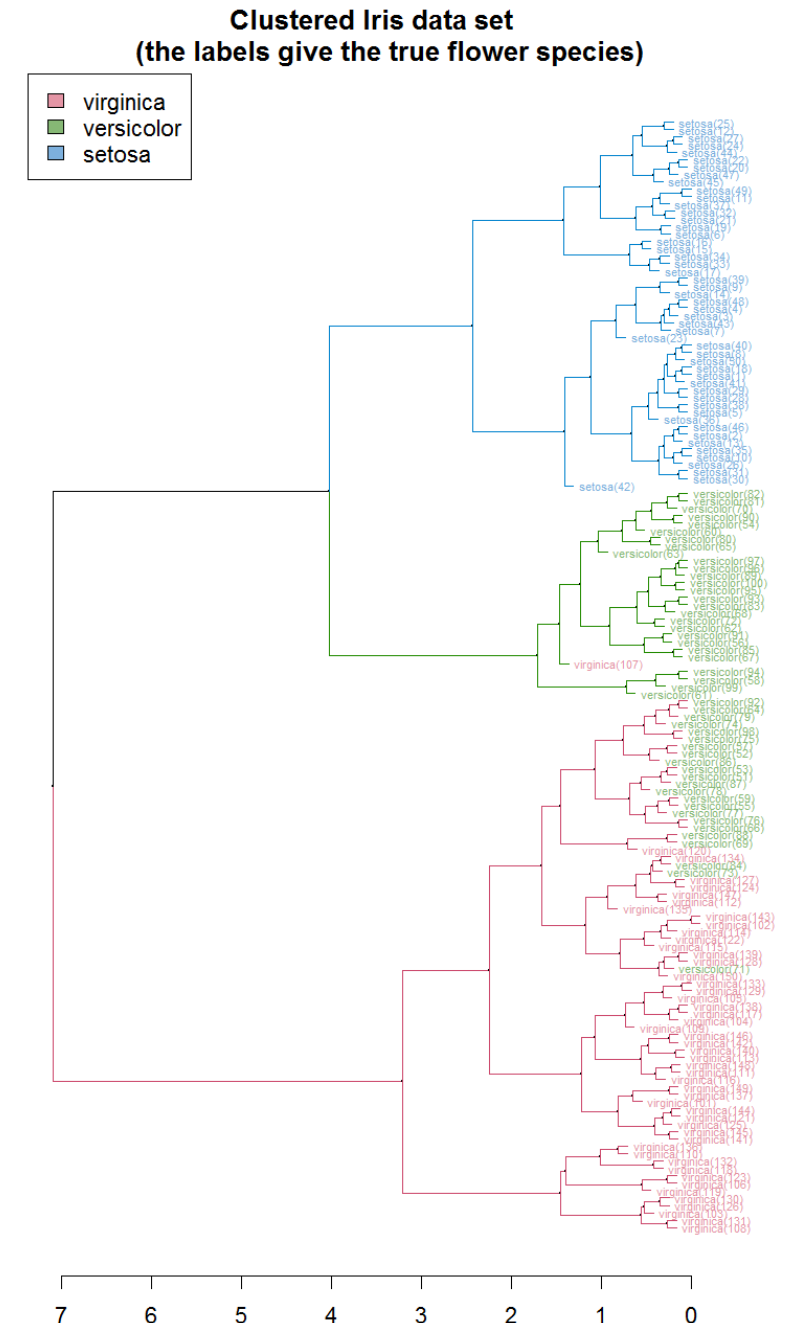
Nenadzorovano učenje



- **lastnosti:**
 - nenadzorovano učenje je **bolj subjektivno** kot nadzorovano učenje, ker nima enoznačnega formalnega cilja kot je "napovedovanje vrednosti odvisne spremenljivke" pri nadzorovanem učenju
 - velikokrat lažje (**cenejše**) **pridobimo neoznačene podatke** (podatke brez odvisne spremenljivke): drage meritve, ekspertno mnenje, globalna ocena (npr. filma)?
- **primeri uporabe:**
 - odkrivanje skupin rakavih bolnikov, grupiranih po različnih rezultatih meritev izraženosti genov,
 - odkrivanje skupin kupcev, grupiranih po njihovi zgodovini brskanja in nakupovanja
 - odkrivanje skupin filmov, grupiranih glede na ocene, podane s strani gledalcev

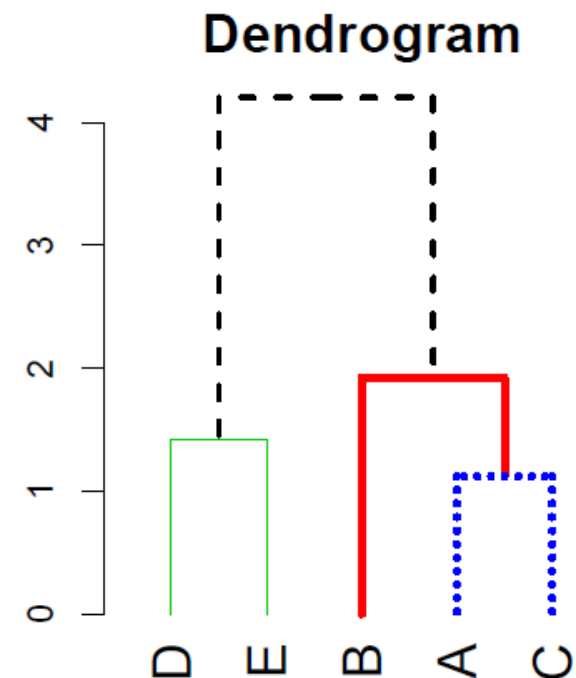
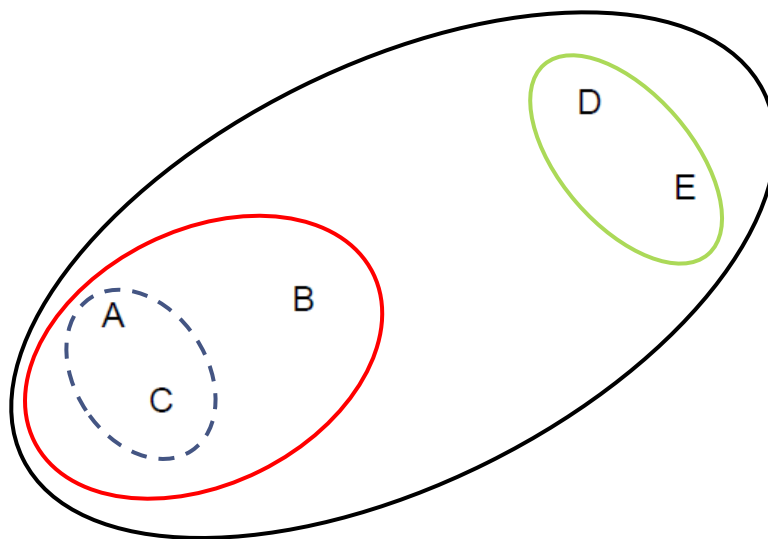
Gručenje

- **gručenje (angl. clustering)** je najbolj uporabljana metoda nenadzorovanega učenja
- **cilj:** iskanje homogenih podskupin v učnih podatkih
- metode:
 - **hierarhično gručenje:** iščemo vnaprej neznano število gruč. Rezultat gručenja je vizualna reprezentacija skupin, imenovana dendrogram, ki nam nudi vpogled v oblikovanje različnega števila gruč
 - **metoda k-means:** optimizacijski algoritem, ki poskuša iterativno primere gručiti v vnaprej podano število k gruč



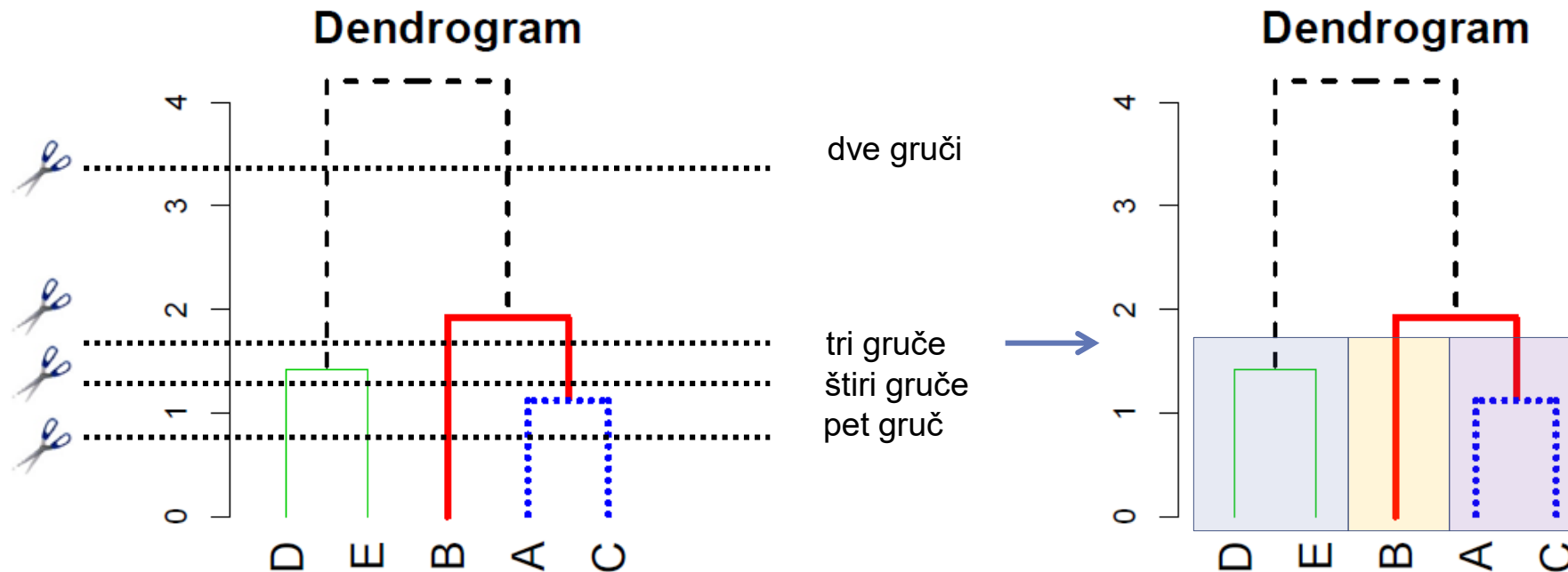
Hierarhično gručenje

- dva pristopa:
 - združevalni** (angl. *agglomerative*): gradnja dendrograma začnši od listov proti korenu s postopkom **združevanja** glede na razdaljo
 - delilni** (angl. *divisive*): gradnja dendrograma od korena proti listom, na vsakem koraku **delimo** gručo na podgruče
- primer združevalnega pristopa:
 - začni z vsako točko v svoji gruči
 - najdi dve najbližji gruči in ju združi
 - ponavljaj, dokler ne združiš vseh gruč



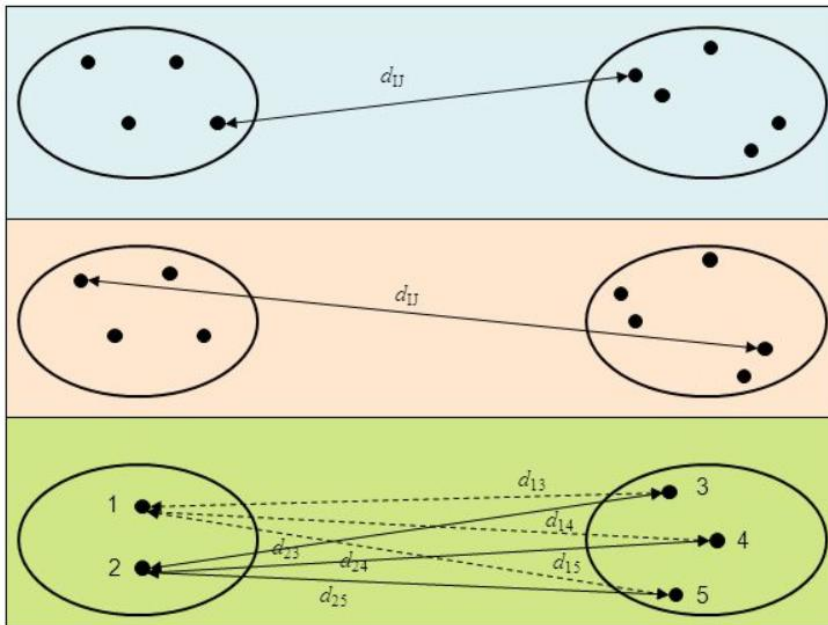
Interpretacija dendrograma

- rezanje dendrograma določi mejo, pri kateri prenehamo z združevanjem gruč
- z rezanjem dendrograma na različnih višinah torej določamo število ciljnih gruč



Merjenje razdalj

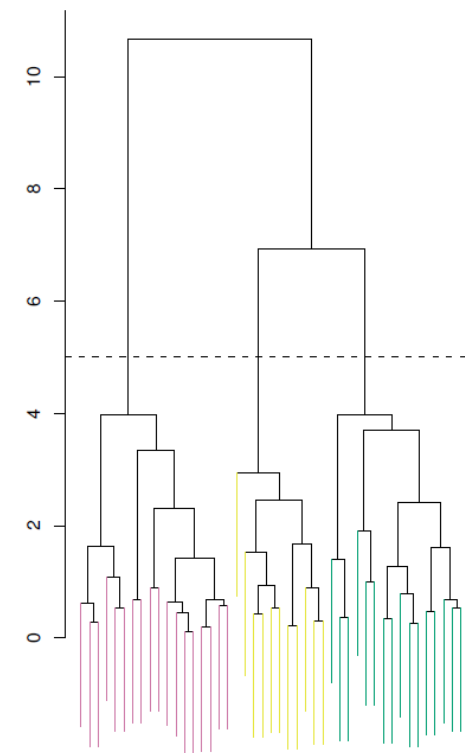
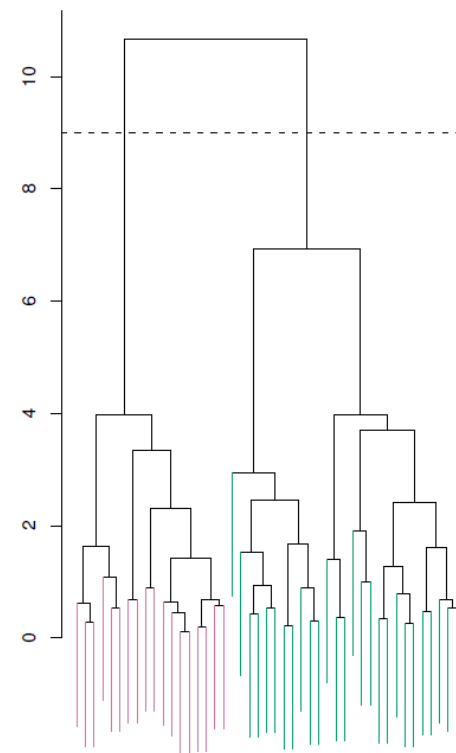
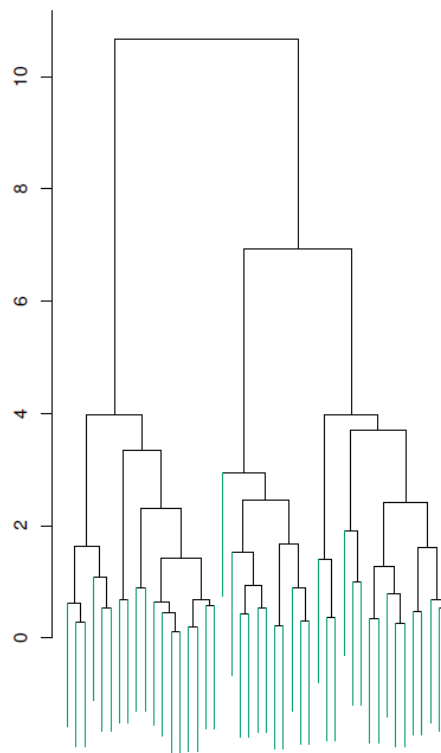
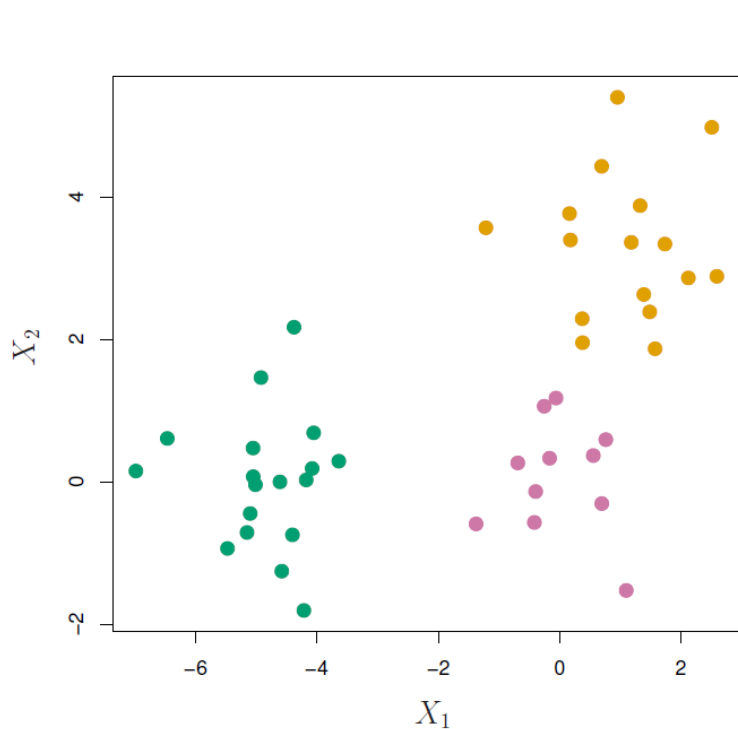
- med učnimi primeri uporabljamo že znane mere za merjenje razdalj (evklidska razdalja, manhattanska razdalja, korelacija med vrednostmi atributov ...)
- posebno obravnavo moramo posvetiti merjenju razdalj:
 - med **posameznim učnim primerom** in **gručo**
 - med **dvema gručama**
- kot razdaljo v teh primerih lahko upoštevamo:



- razdaljo med **najbližjima** primeroma (enojna povezanost, angl. *single linkage*)
$$d(C_1, C_2) = \min_{i,j} \{d_{ij} | i \in C_1, j \in C_2\}$$
- razdaljo med **najbolj oddaljenima** primeroma (popolna povezanost, angl. *complete linkage*)
$$d(C_1, C_2) = \max_{i,j} \{d_{ij} | i \in C_1, j \in C_2\}$$
- **povprečno razdaljo** med vsemi primeri (povprečna povezanost, angl. *average linkage*)
$$d(C_1, C_2) = \sum_{i \in C_1, j \in C_2} \frac{d_{ij}}{|C_1||C_2|}$$

Primer

- 45 primerov, 2 atributa (oznaka razreda je skrita pred algoritmom za gručenje)
- uporabljena evklidska razdalja in merjenje razdalj s polno povezanostjo (complete linkage)
- dendrogram prikazuje rezanja na različnih višinah



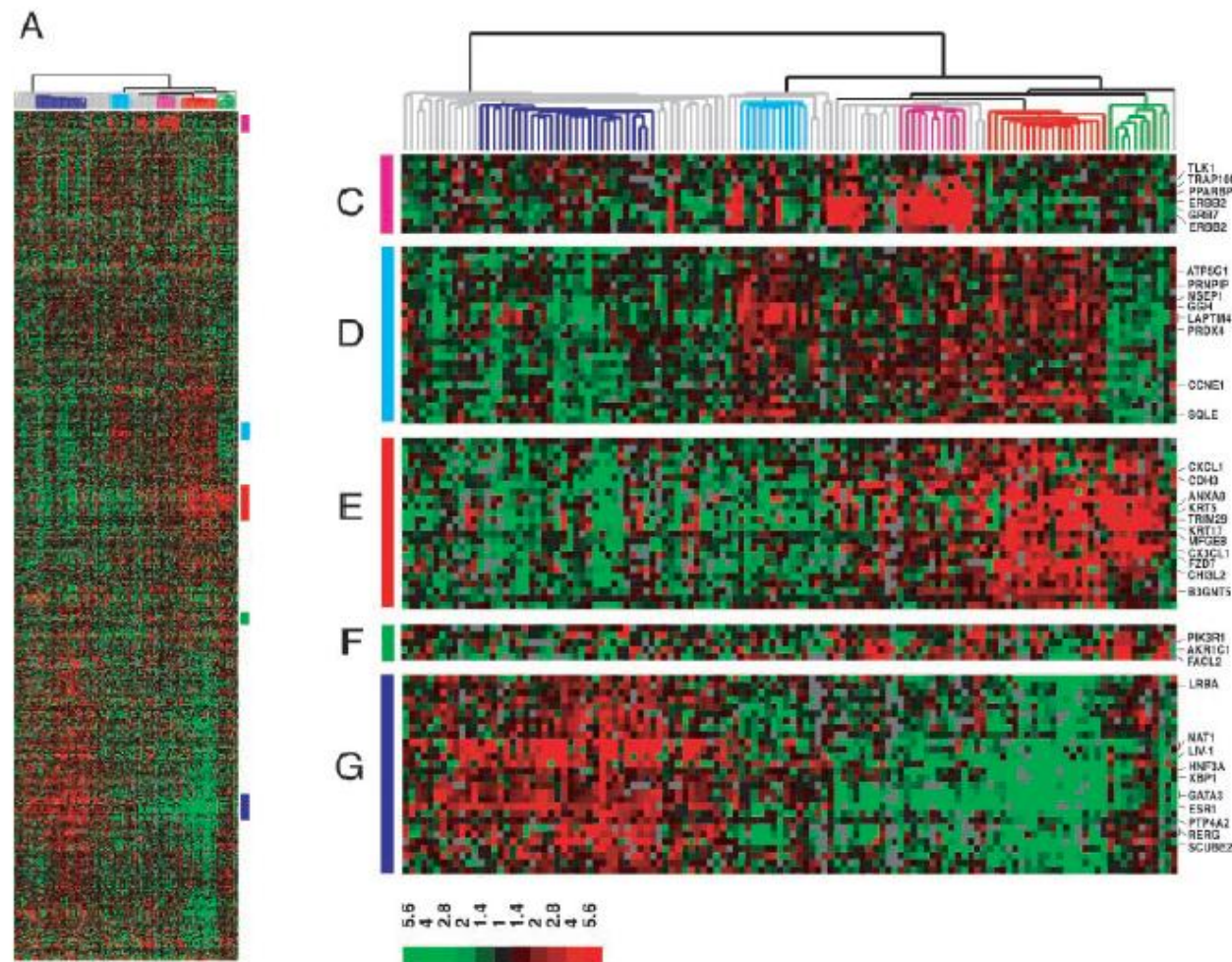
Opombe

- **normalizacija atributov** (glej desno sliko →)
- **časovna zahtevnost:**
 - združevalni pristop: $O(n^2 \log n)$:
 n^2 časa za izračun matrike razdalj, $\log n$ za urejanje razdalj
 - delilni pristop: $O(2^n)$:
za iskanje optimalne delitve na dve podgruči
- **parametri**
 - katero mero razdalje izbrati?
 - kateri pristop merjenja razdalj med gručami izbrati?
 - kolikšno naj bo ciljno število gruč?



Primer uporabe

- analiza različnih vrst tumorja na prsih
glede na izraženost genov
Sørli, Therese, et al. "Gene expression patterns of breast carcinomas distinguish tumor subclasses with clinical implications." Proceedings of the National Academy of Sciences 98.19 (2001): 10869-10874.
- mera razdalje: korelacija
- razdalja med gručami: povprečna razdalja med primeri
- atributi: izraženost 500 genov
- rezultati:
 - identifikacija sorodnih skupin pacientov
 - identifikacija izraženih genov v skupinah pacientov



Izpitna naloga

- 2. izpitni rok, 13. 2. 2019 (prilagojena naloga)

4. NALOGA (10t):

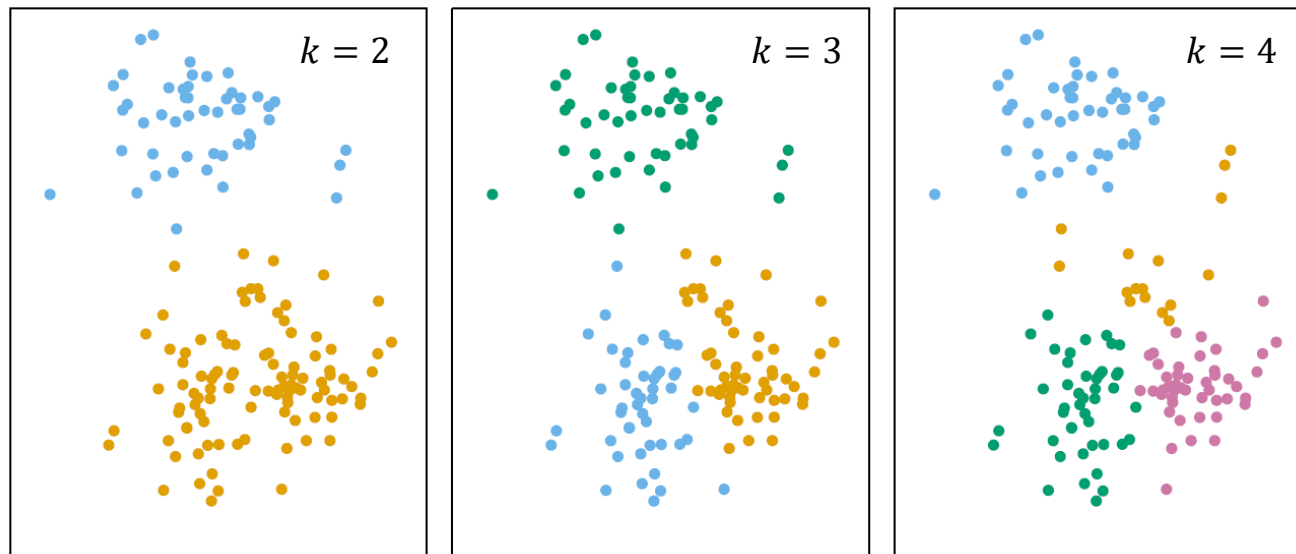
Podanih je pet točk z vrednostmi atributov X in Y, ki predstavljata koordinati na grafu.

- a) (5t) Izvedi algoritem hiearhičnega razvrščanja naštetih točk in nariši dendrogram. Uporabi Manhattansko razdaljo in pristop popolne povezanosti (angl. complete linkage) merjenja razdalj med gručami.
- b) (3t) Dendrogram iz prejšnje naloge poreži tako, da dobimo dve gruči.

točka	X	Y
A	1	1
B	3	1
C	1	3
D	3	2
E	4	3

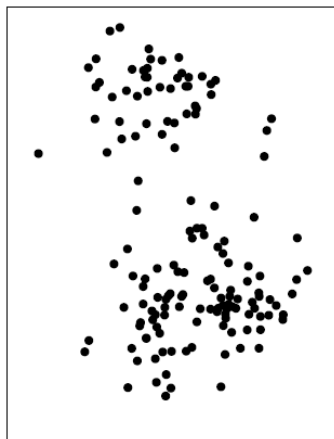
Metoda voditeljev

- angl. *k-means clustering*
- postopek:
 - izberi **začetno število gruč k** in **naključno priredi** vsak učni primer eni od gruč
 - ponavljaj do konvergence (dokler se pripadnost gručam spreminja):
 - za vsako izmed k gruč izračunaj **centroid** (točka, določena s srednjo vrednostjo atributov vseh primerov, ki pripadajo gruči)
 - spremeni pripadnost vsakega primera tisti gruči, katere centroid je najbližji (glede na izbrano mero razdalje)
- primer: različne rešitve za $k = 2$, $k = 3$ in $k = 4$

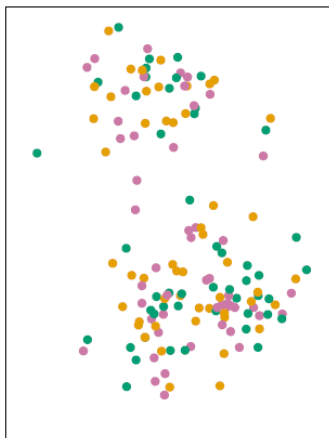


Primer izvajanja

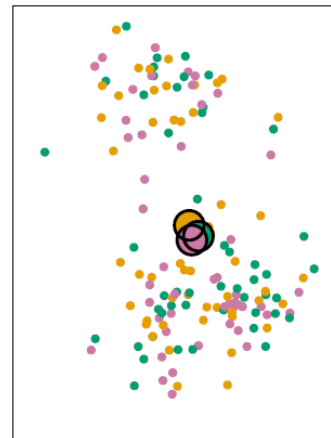
učna množica



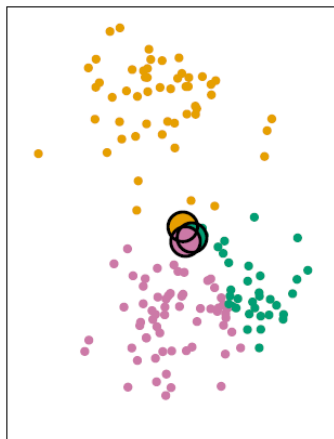
naključna določitev
gručam



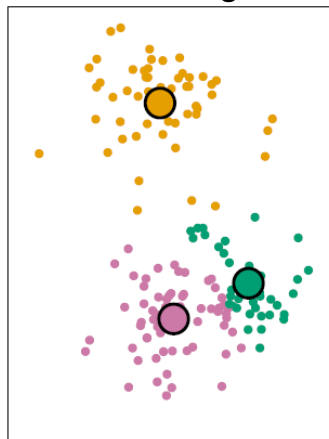
izračun začetnih
centroidov gruč



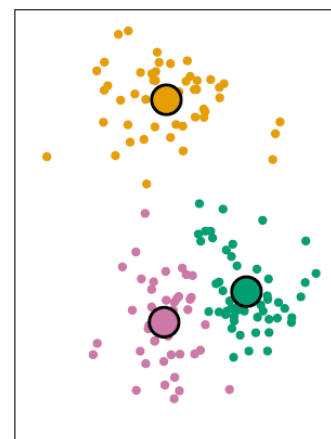
določitev pripadnosti primerov
najbližjemu centroidu



ponovni izračun
centroidov gruč

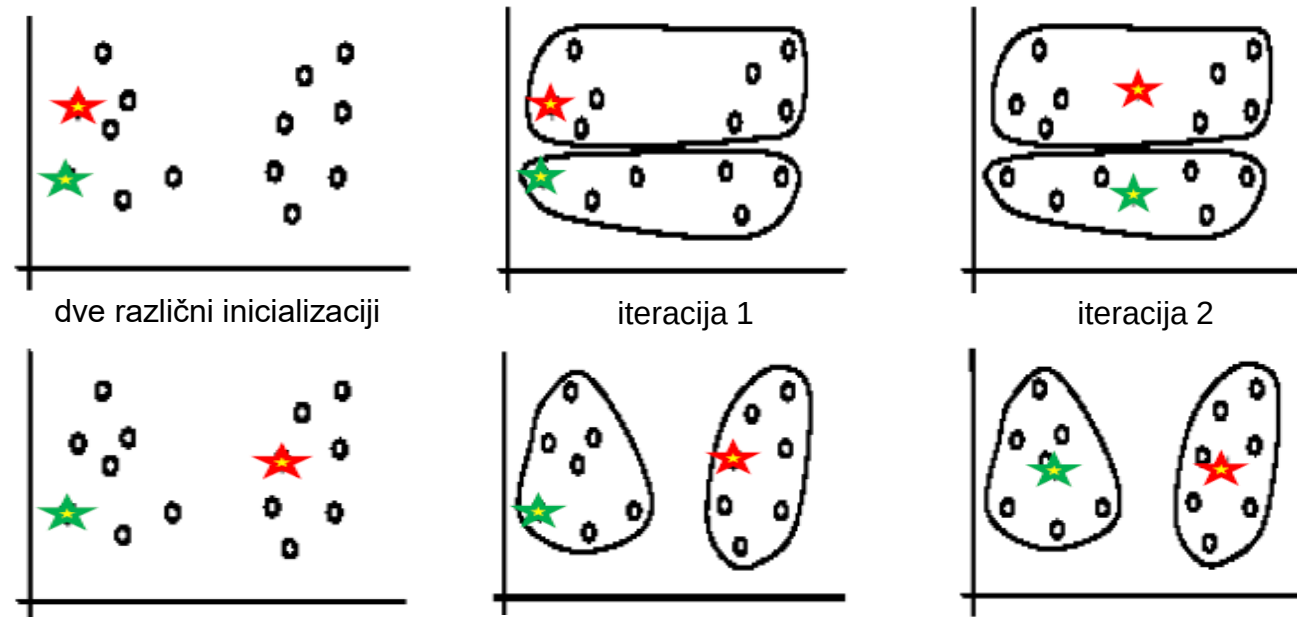


rezultat po 10 iteracijah



Lastnosti algoritma

- metoda na vsakem koraku znižuje varianco znotraj gruč (s prerazporejanjem pripadnosti primerov najbližji gruči)
- algoritem ne najde globalnega optima (glede na cilj minimizacije variance znotraj gruč), rešitev je odvisna od začetne inicializacije
- izračun centroidov je potrebno prilagoditi, če so atributi diskretni
- algoritem je občutljiv na šum (angl. *outliers*)
- težka objektivna evalvacija, vendar uporabno v praksi





Preiskovanje